

Empirical Modeling of Solar Influence on Polar Cap Ionospheric Electron Temperature

All the code discussed can be found in `/data1/small/samikris/Resources`.

Introduction

The goal of my project was to develop an empirical model relating electron temperature in the polar cap ionosphere to solar conditions (SZA and F10.7). I've spent the past few months processing RISR-N data and visualizing it to examine the influence of SZA and F10.7 on electron temperature. To create reduced files, I used a similar structure to Mike Lundquist's scripts for his project observing electron density in the ionosphere.

Data Processing Python Script

The goal of this script was to create reduced hdf5 files with only the information needed to understand T_e in the F-region.

Data contained in files:

- Median electron temperature
- T_e error
- Times
- Altitudes
- Filtered electron temperature
- Filtered error
- Solar zenith angle (filtered by beam)
- Neutral temperature
- F107
- F107a

For one observation, there were multiple T_e measurements for each timestamp, so I calculated the simple median. This ensured that each observation date only had one corresponding T_e value. The final script is in the Resources directory, titled `ReducingFiles.py`. It only creates reduced files for the 1 minute integration time. When you run it in the command line, you would specify the script name and the range of dates you want to create reduced files for (`python ReducingFiles.py 03142015 03152015`).

Filtering Process:

1. Get temperatures within the 250-400 km altitude range
2. Remove measurements with high uncertainty by taking errors below 400
3. Extract temperatures from 0 - 5000 K
4. Take median across altitude range and calculate propagated error

Key Sections

- Finding the beam closest to 75 degrees vertical

```
#Find the beam that is closest to 75 degrees vertical
bplot = np.argmax(file_array['/']['BeamCodes'][:,2] % 75.0)
```

- Getting Electron Temperature (Te) and Error (dTe) from 250 - 400 km

```
#Getting valid altitudes
goodalts = np.logical_not(np.isnan(file_array['/FittedParams']['Altitude'][bplot,:]))
altitudes = file_array['/FittedParams']['Altitude'][bplot, goodalts]
```

```
#Get electron temperature and its error
Te = file_array['/FittedParams']['Fits'][:, bplot, :, -1, 1]
dTe = file_array['/FittedParams']['Errors'][:, bplot, :, -1, 1]
```

```
#Find indices for altitude range of interest
a1 = np.argmin(np.abs(altitudes[:] - min_altitude))
a2 = np.argmin(np.abs(altitudes[:] - max_altitude))
```

```
#Filter Te and dTe by altitude range
Te_filt = Te[:, a1:a2]
dTe_filt = dTe[:, a1:a2]
```

- Taking errors below 400, temperatures from 0 - 5000 K, and then calculating the median

```
#Apply error threshold (flagging bad data)
error_threshold = 400
badindex = np.where((dTe_filt > error_threshold) | (Te_filt <= 0) | (Te_filt >= 5000)) #I forgot this in my first run!
Te_filt[badindex] = np.nan #maintaining array size, just filling with nan values
```

```
#Calculate median Te across altitude range
Te_med = np.nanmedian(Te_filt, axis=1)
```

- Calculating the propagated error (because I'm taking the median) using the formula:

```
#Calculate propagated error
N = np.sum(~np.isnan(dTe_filt), axis=1) #Count of non-NaN values for each time
prop_errors = np.sqrt(np.nansum(dTe_filt**2, axis=1) / np.maximum(N, 1)) #Avoid division by zero
```

- Extract SZA, Tn, f107, f107a

```
#Filter out sza and Tn to match altitude range
sza_filt = file_array['/MSIS']['SolarZen'][:, bplot, a1:a2]
Tn = file_array['/MSIS']['Tn'][:, bplot, a1:a2]
f107 = file_array['/MSIS']['f107'][:, :]
f107a = file_array['/MSIS']['f107a'][:, :]
```

Complete Data in a Notebook

This code collects all the valid data points from multiple processed files within each year and combines them into single yearly arrays. Each time it reads a new file, it filters out NaN values and appends the valid data to the growing yearly arrays.

By the end, yearly_data[year] contains all the electron temperature, solar zenith angle, neutral temperature, F10.7 flux, and time data for that entire year, aggregated from all the processed files in that year's directory. Then, larger arrays that combine the data from all the years are created so the data can be visualized more easily.

```

for year_folder in year_folders:
    try:
        year = int(year_folder)
        year_path = os.path.join(base_dir, year_folder)
        print("processing data for:", year)

        #Creating arrays
        year_te = np.array([])
        year_sza = np.array([])
        year_time = np.array([])
        year_f107 = np.array([])
        year_tn = np.array([])

        processed_files = [f for f in os.listdir(year_path) if f.startswith('processed-')]

        for filename in processed_files:
            filepath = os.path.join(year_path, filename)

            try:
                with h5py.File(filepath, 'r') as data_file:
                    #Extract data and add to array
                    median_te = data_file['/FittedParams/MedianTe'][:]
                    unix_times = data_file['/FitInfo/Times'][:, 0]

                    sza = data_file['/FitInfo/SolarZenithAngle'][:,]
                    tn = data_file['/FitInfo/Tn'][:,]
                    f107 = data_file['/FitInfo/f107'][:,]

                    #Extract the middle column for SZA
                    sza_middle = sza[:, 3]
                    tn_middle = tn[:, 3]

                    #Filter out NaN values
                    valid_indices = ~np.isnan(median_te)

                    if np.any(valid_indices):
                        #Concatenate to year arrays
                        year_te = np.concatenate([year_te, median_te[valid_indices]])
                        year_sza = np.concatenate([year_sza, sza_middle[valid_indices]])
                        year_tn = np.concatenate([year_tn, tn_middle[valid_indices]])
                        year_f107 = np.concatenate([year_f107, f107[valid_indices]])
                        year_time = np.concatenate([year_time, unix_times[valid_indices]])
                    except Exception as e:
                        print(f" Error processing {filename}: {e}")

                #Store this year's data
                yearly_data[year] = {
                    'te': year_te,
                    'sza': year_sza,
                    'time': year_time,
                    'f107': year_f107,
                    'tn': year_tn
                }
                print(f" Collected {len(year_te)} data points for {year}")

            except ValueError:
                print(f"Skipping folder {year_folder} - not a year folder")

```

```

                    if np.any(valid_indices):
                        #Concatenate to year arrays
                        year_te = np.concatenate([year_te, median_te[valid_indices]])
                        year_sza = np.concatenate([year_sza, sza_middle[valid_indices]])
                        year_tn = np.concatenate([year_tn, tn_middle[valid_indices]])
                        year_f107 = np.concatenate([year_f107, f107[valid_indices]])
                        year_time = np.concatenate([year_time, unix_times[valid_indices]])
                    except Exception as e:
                        print(f" Error processing {filename}: {e}")

                #Store this year's data
                yearly_data[year] = {
                    'te': year_te,
                    'sza': year_sza,
                    'time': year_time,
                    'f107': year_f107,
                    'tn': year_tn
                }
                print(f" Collected {len(year_te)} data points for {year}")

            except ValueError:
                print(f"Skipping folder {year_folder} - not a year folder")

```

```

#trying to combine all the years
all_years_te = np.array([])
all_years_sza = np.array([])
all_years_time = np.array([])
all_years_f107 = np.array([])
all_years_tn = np.array([])
all_years = np.array([]) #To keep track of which year each point belongs to

for year, data in yearly_data.items():
    all_years_te = np.concatenate([all_years_te, data['te']])
    all_years_sza = np.concatenate([all_years_sza, data['sza']])
    all_years_time = np.concatenate([all_years_time, data['time']])
    all_years_f107 = np.concatenate([all_years_f107, data['f107']])
    all_years_tn = np.concatenate([all_years_tn, data['tn']])

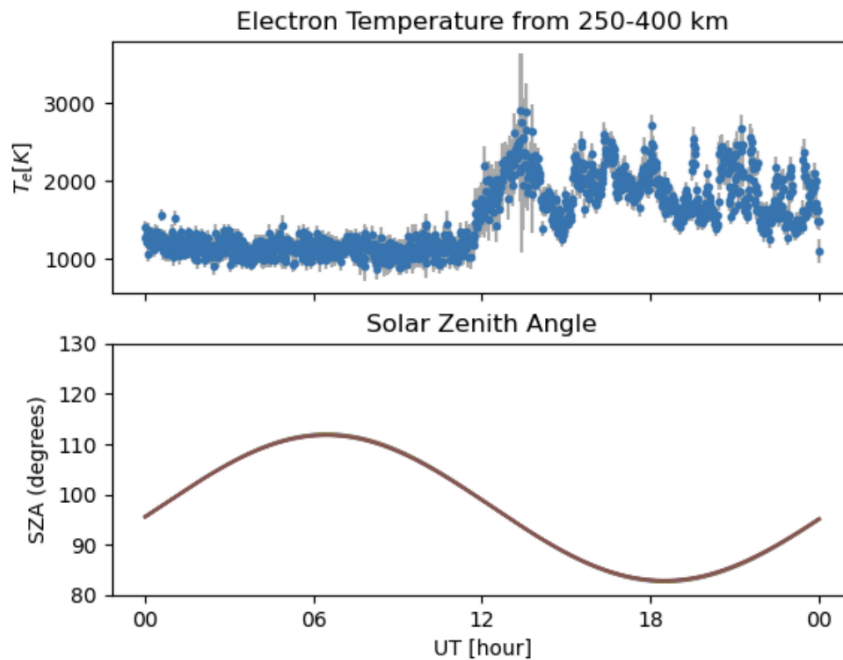
    year_labels = np.full(len(data['te']), year)
    all_years = np.concatenate([all_years, year_labels])

print(f"total data points: {len(all_years_te)}")

```

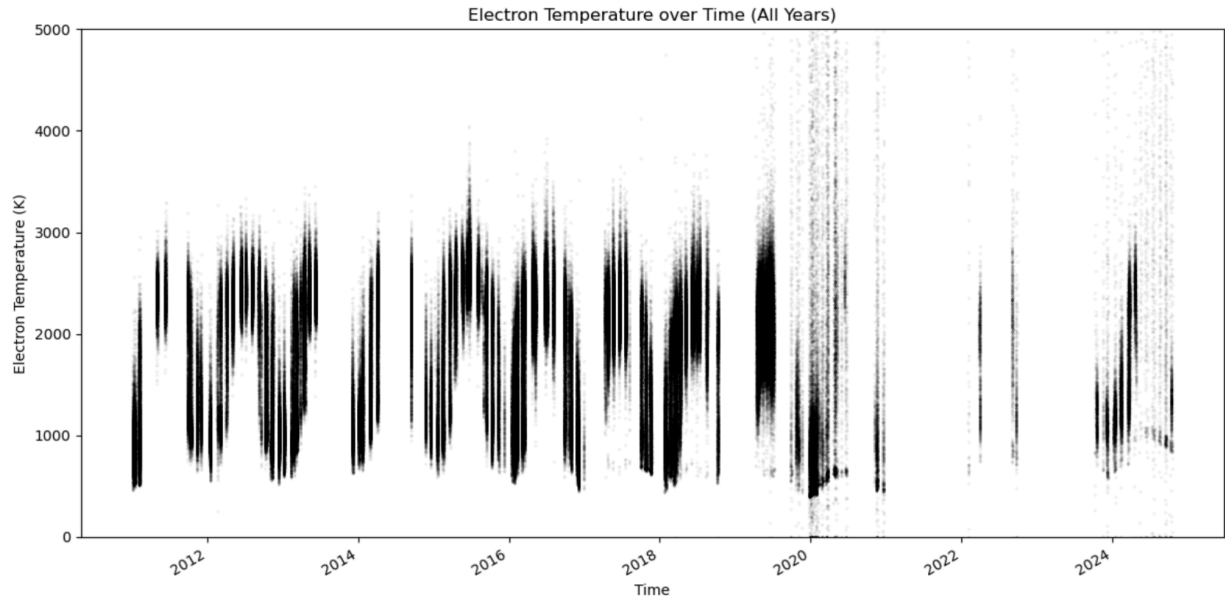
Key Plots

Single Observation of Te and SZA



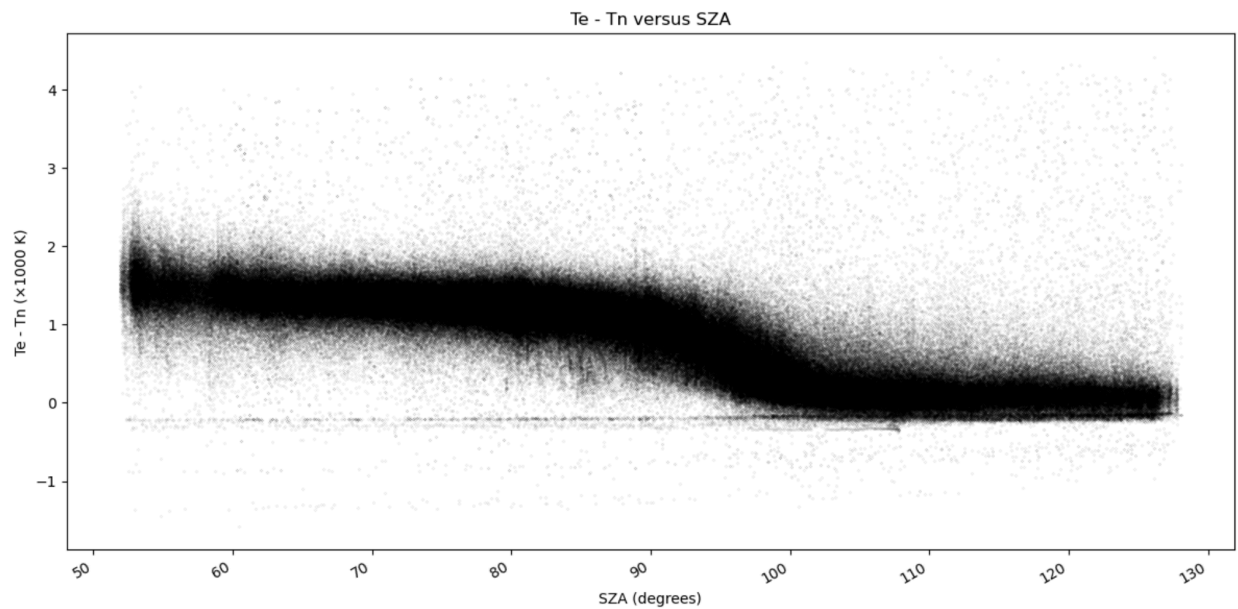
All Years Te

No clear trend, but helpful for understanding the time period of data examined and temperature range.



Te - Tn versus SZA

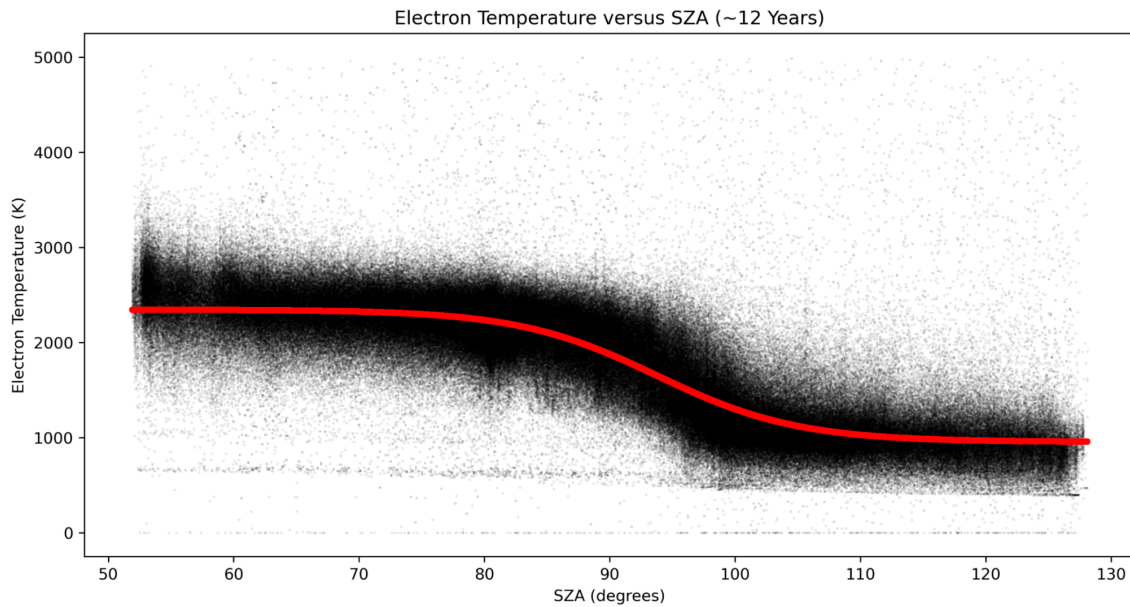
Reveals the strong dependence between SZA and Te



Curve Fits

Regular Tanh Fit

Using scipy's optimize feature, I initially fit a regular arctanh to the data, which somewhat captured the transition between day and night, but appeared to be very symmetric.



This was the function the model fit parameters for:

$$Te_{model} = a \tanh\left(\frac{(SZA - b)}{c}\right) + d$$

The final parameter where the transition where ionospheric sunset occurs is at ~93.42 degrees.

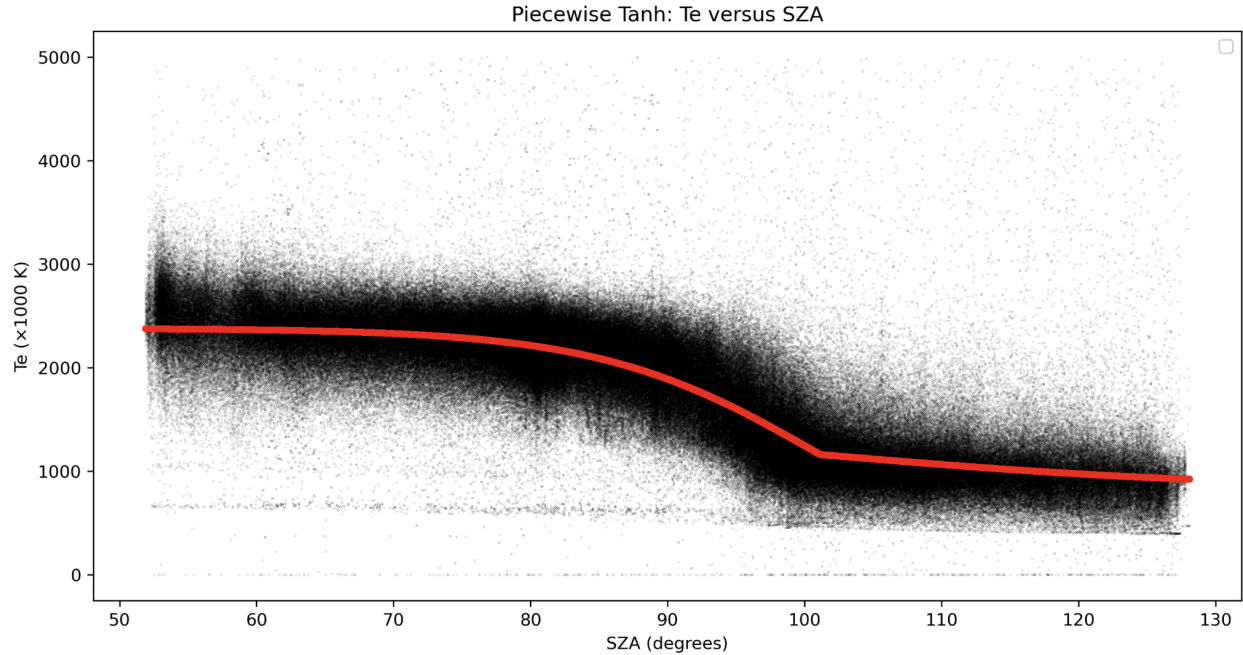
Asymmetric Tanh Fit

This model did work, but the dayside was much more sinusoidal, while the night side was essentially flat. To capture this relationship and the difference in amplitude of both sides of the curve, we used an asymmetric tanh function.

$$Te_{left} = a_1 \tanh\left(\frac{(SZA - b)}{c_1}\right) + d$$

$$Te_{right} = a_2 \tanh\left(\frac{(SZA - b)}{c_2}\right) + d$$

This found the parameters for two tanh functions that met at the “d” parameter, or transition, but had different shapes. The result was a much better physical representation of the data.



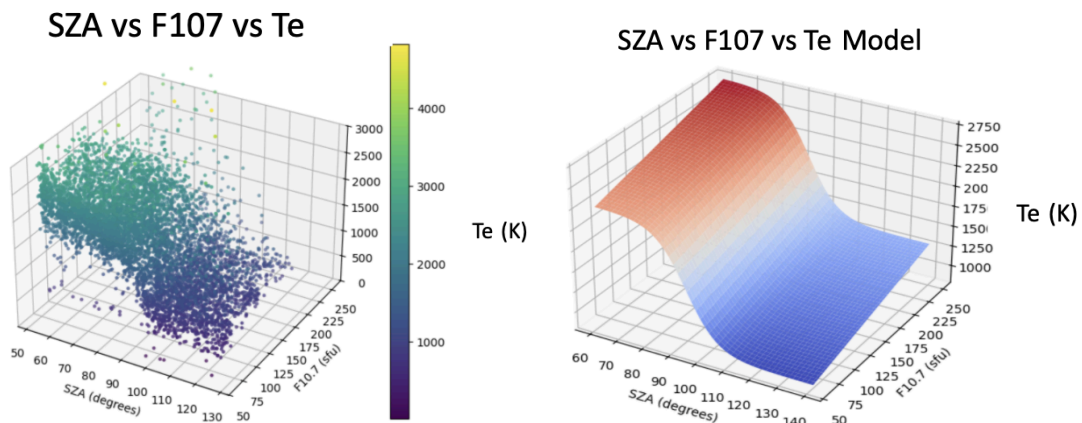
The final parameter where the transition where ionospheric sunset occurs is at ~ 101.12 degrees.

Surface Fits

To observe the relationship between SZA, F107, and Te, a surface was fit over the 3D data. The function was of the form:

$$Te_{model} = a \tanh\left(\frac{(SZA - b)}{c}\right) + d(F10.7) + e$$

While a strong dependency between SZA and Te is the predominant relationship, there is a slight positive, linear correlation between F107 and Te. This could indicate that there is some additional heating from increased solar activity.



Final Summary

So far, a key finding was that solar zenith angle is a dominant factor controlling electron temperature. In the future, it would help to further isolate the relationship between F10.7 and electron temperature. Once the data has been detrended for F10.7 and SZA, the next step would be to investigate the influence of solar wind parameters.

Other notes:

- After plotting the data, I realized that I forgot to actually set a filter for temperature values between 0 to 5000 K. A new line has been written to account for that if the files need to be reprocessed.
- In the testing folder, there are some scripts where I tried to run the process in parallel - the time difference running parallel was pretty small, so it didn't make a major difference in terms of efficiency.